SIMATS ENGINEERING ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking **COURSE CODE: ITA1408**

COURSE OUTCOME COVERED

CO4: *Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.*

QUESTIONS: 01 **Total Marks:100**

Annexure A

SECURITY TESTING SIMULATION TASK

Code of Conduct:

I ***A.Bhavya – 192424417*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.Bhavya

Annexure A

Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and webserver vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

Objective

To assess an authorized web server by collecting basic network information, identifying available ports and services, detecting the technologies used by the web server, analyzing HTTP headers, scanning for security weaknesses, and locating accessible directories and files.

Target :

- **Target IP:** 192.168.56.102
- **Target:** Metasploitable 2

Tools Used :

Ping, Traceroute, Netstat, Nmap, WhatWeb, Curl, Wget, Nikto, Gobuster and Dirb.

Procedure and Commands

1. **Network Reconnaissance :** The initial step was performed to verify whether the target machine was accessible and to obtain basic network information.

Command 1: ping -c 4 192.168.56.102

Observation:

The Ping command successfully reached the target and showed **0% packet loss**, confirming that the target system was responding to network requests.

Command 2: traceroute 192.168.56.102

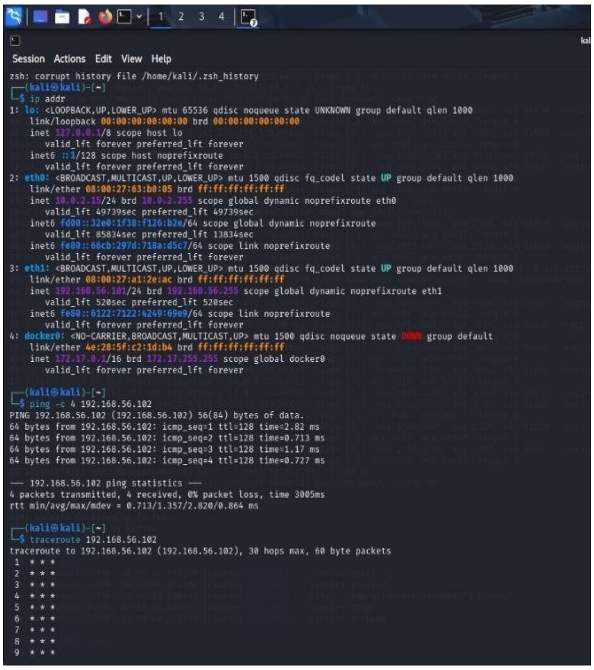
Observation:

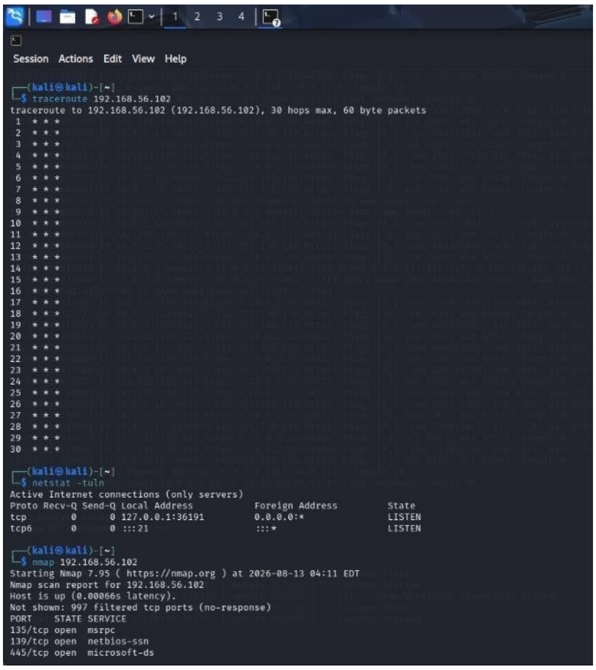
Traceroute was used to identify the network path followed by packets from the Kali Linux machine to the target server.

Command 3: netstat -tuln

Observation:

Netstat displayed the listening network connections and ports available on the local system.





2. **Port and Service Identification :** Nmap was used to determine which ports were accessible on the target and to identify the services associated with them

Command 1: nmap 192.168.56.102
Command 2 : nmap -sV 192.168.56.102

Observation:

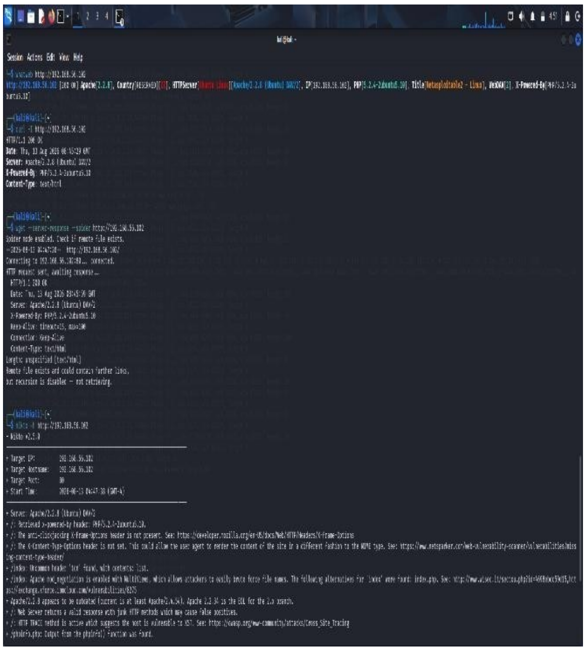
The Nmap scan identified multiple open ports and services. Ports such as 135, 139, and 445 were observed, which were associated with services including MSRPC, NetBIOS, and Microsoft-DS/SMB.

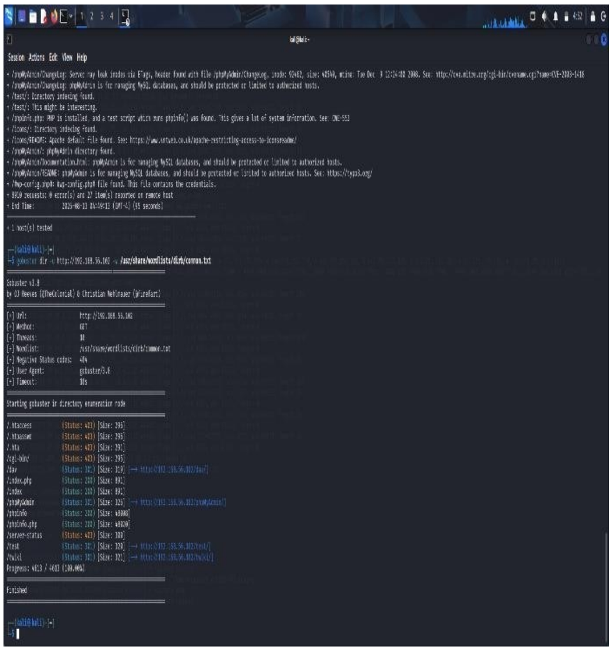
3. **Web Technology Identification :**

Command 1: whatweb http://192.168.56.102

Observation:

The scan detected Apache 2.2.8, Ubuntu Linux, PHP 5.2.4, and WebDAV**.** The web server responded with an HTTP 200 OK status.





4. **HTTP Header Analysis :** The HTTP response headers were inspected to identify information that could reveal details about the server configuration.

Command 1: curl -I http://192.168.56.102
Command 1: wget --server-response --spider http://192.168.56.102

Observation:

The responses exposed details related to the Ubuntu platform, and PHP version. Such information can help an attacker identify the server software and its possible weaknesses. 5. **Web Vulnerability Scanning :** nikto -h http://192.168.56.102

Observation:

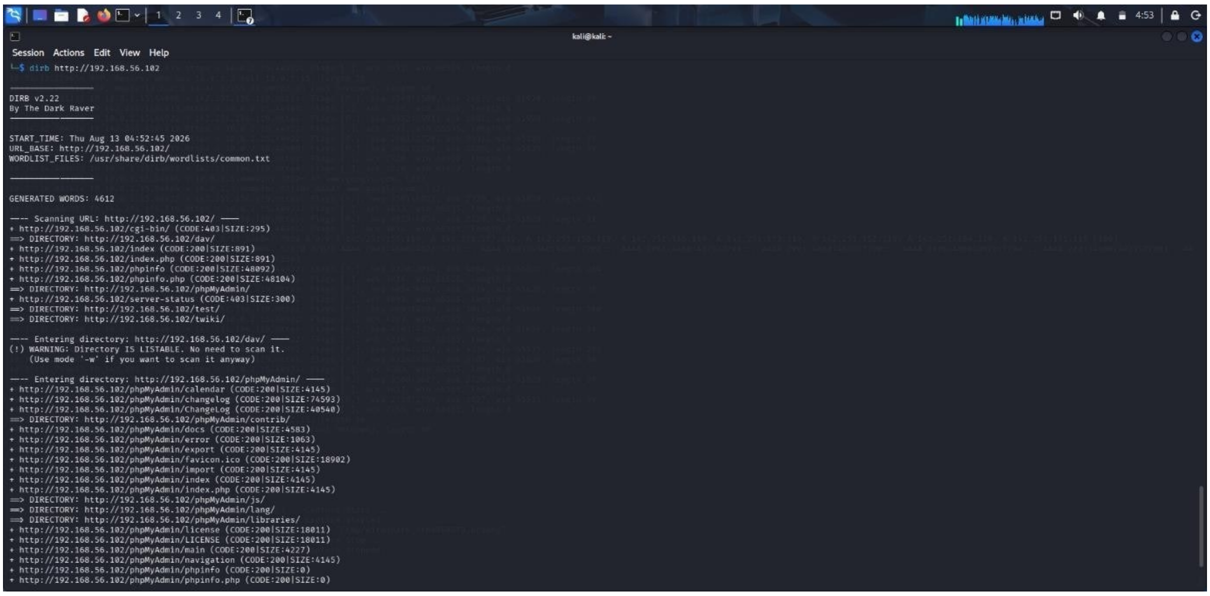
Nikto examined the web server for common configuration weaknesses, outdated software components, exposed resources, and other potentially vulnerable areas.

6. **Directory and File Discovery :** Gobuster and Dirb were used to discover directories and files that may not be directly accessible through the main webpage.

   a. gobuster dir -u http://192.168.56.102 -w /usr/share/wordlists/dirb/common.txt
   b. dirb http://192.168.56.102

Observation:

The directory enumeration tools identified accessible web directories and files that could provide additional information about the server and its configuration.



Observations :

- The target was reachable from Kali Linux.
- Several network services were exposed.
- Apache and PHP versions were disclosed.
- WebDAV was detected.
- HTTP response headers revealed server technology information.

Findings and Severity :

Finding	Severity
Server/PHP version disclosure	Low
Outdated Apache/PHP versions	High
WebDAV exposure	Medium
Exposed directories/resources	Medium
Unnecessary network services	Medium

Recommendations

- Perform regular vulnerability assessments and server hardening to identify and resolve security weaknesses.
- Hide unnecessary server and framework version information.
- Disable WebDAV if it is not required.
- Remove unnecessary services and close unused ports.
- Configure appropriate HTTP security headers.
- Regularly perform vulnerability scanning and server hardening.

Conclusion

The reconnaissance and vulnerability assessment provided an overview of the target web server's network exposure, services, technologies, HTTP information, and accessible web resources. The results indicate that outdated software, information disclosure, WebDAV exposure, and unnecessary services should be addressed to improve the overall security of the server.

Question 2: Web Application Vulnerability Assessment

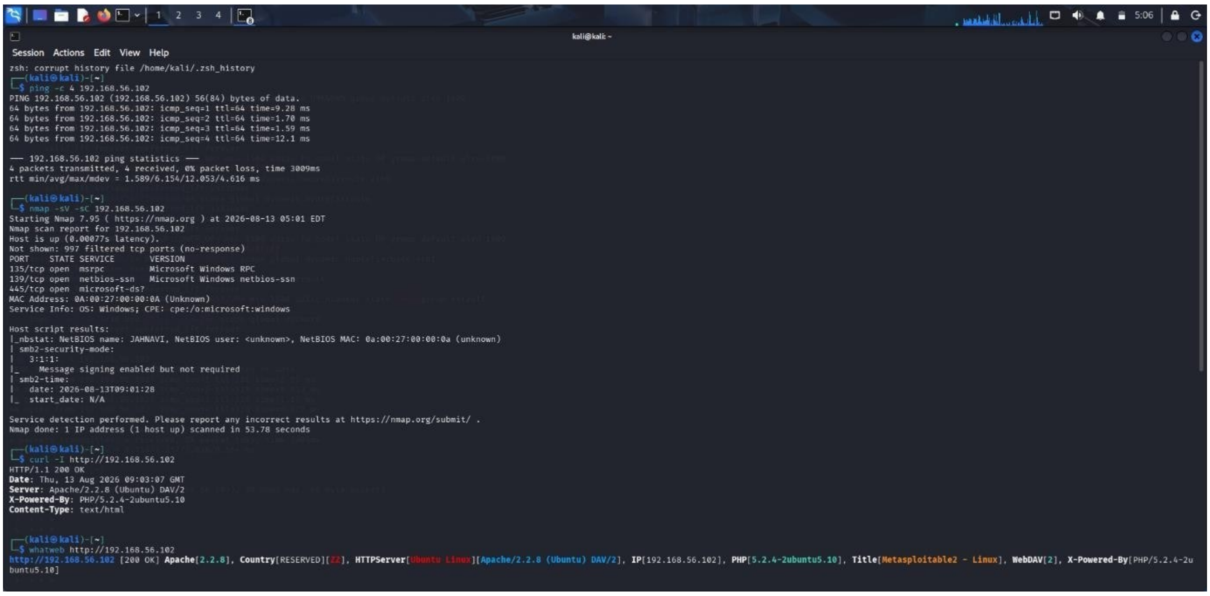
A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

Aim: To perform a vulnerability assessment of an authorized web application using Nmap, Nikto, Burp Suite, and OWASP ZAP in a controlled laboratory environment.

- **Target and Tools :** 192.168.56.102
- Application: Damn Vulnerable Web Application (DVWA)
- Operating System: Metasploitable 2 / Linux
- Tools Used: Nmap, WhatWeb, Nikto, Burp Suite, OWASP ZAP

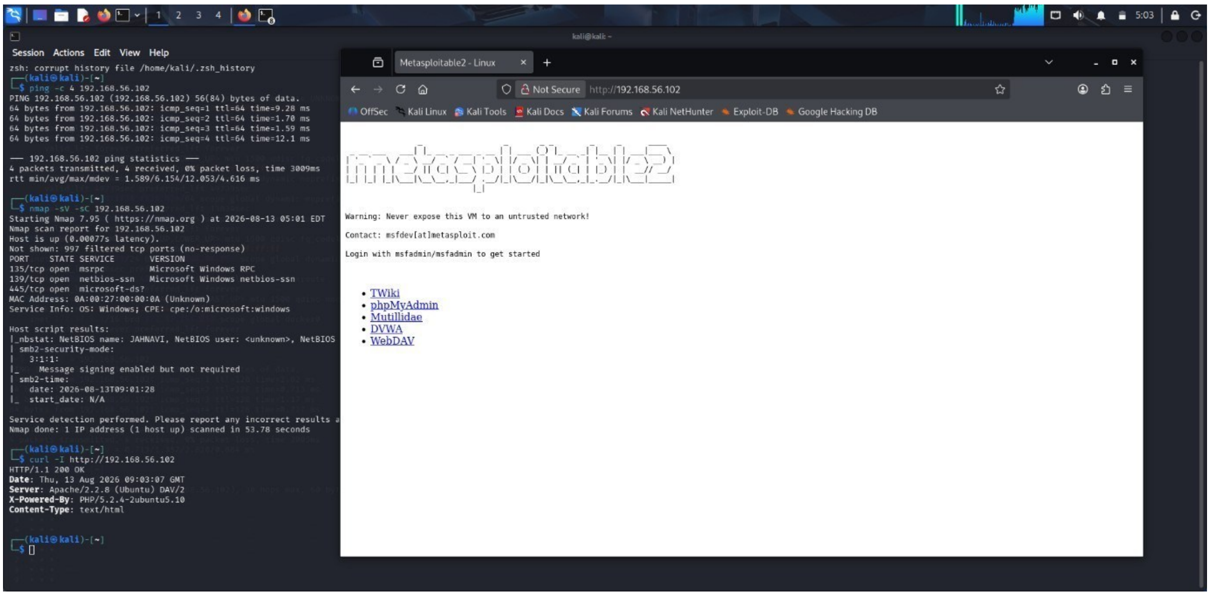
1. Network and Service Identification

- ping -c 4 192.168.56.102
- nmap -sV -sC 192.168.56.102



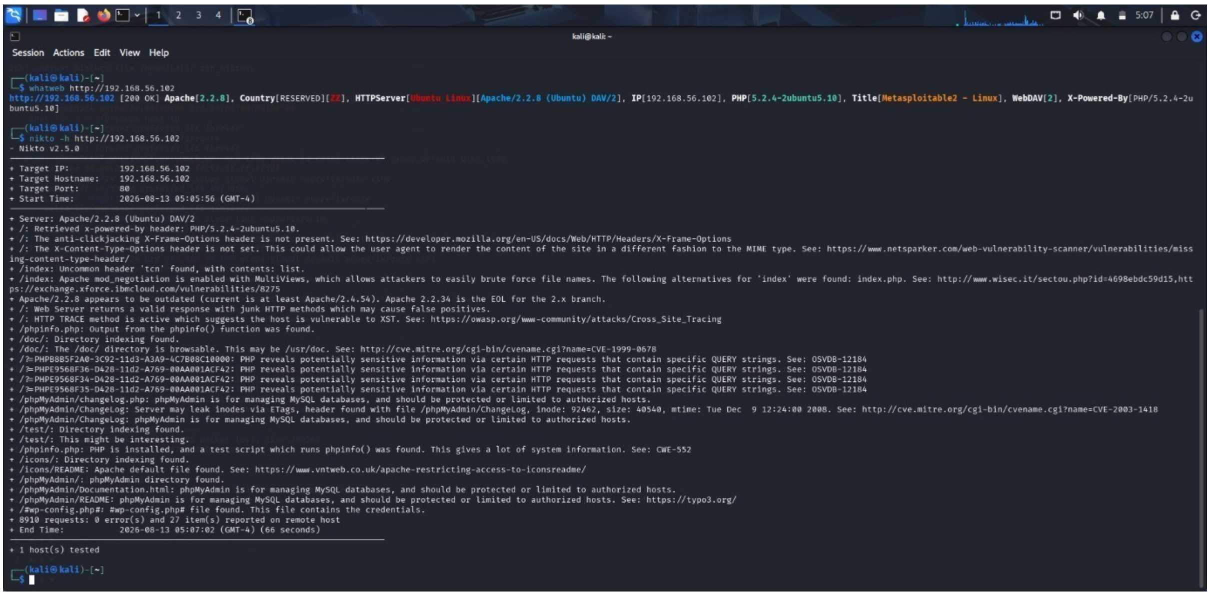
2. Web Technology Identification

- curl -I http://192.168.56.102
- whatweb http://192.168.56.102

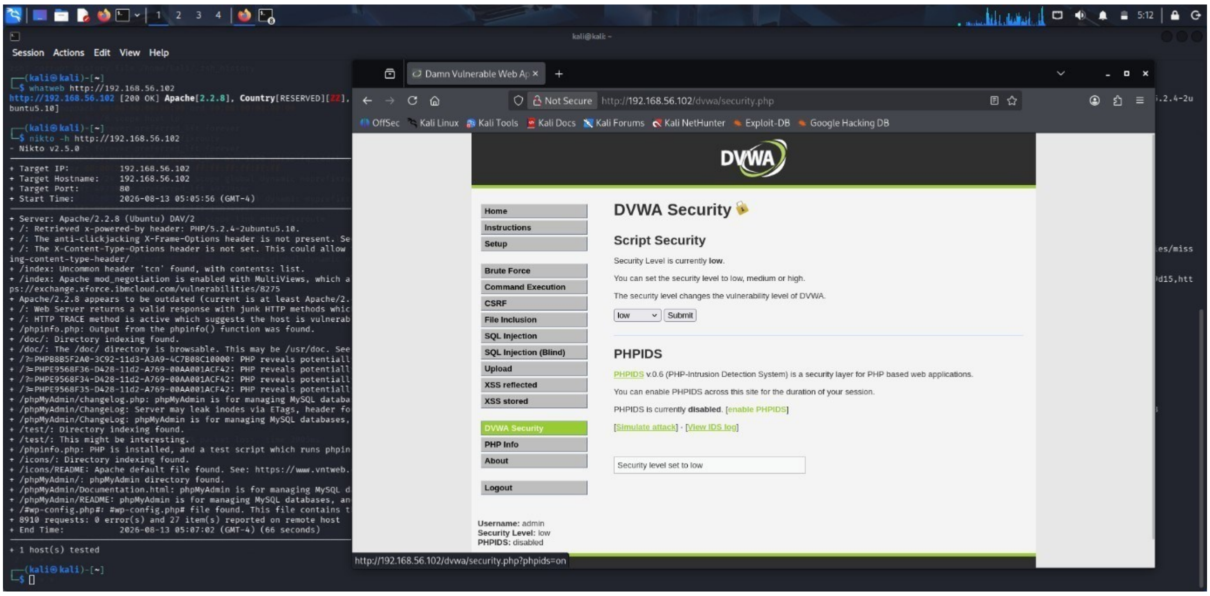


3. Nikto Web Server Scan

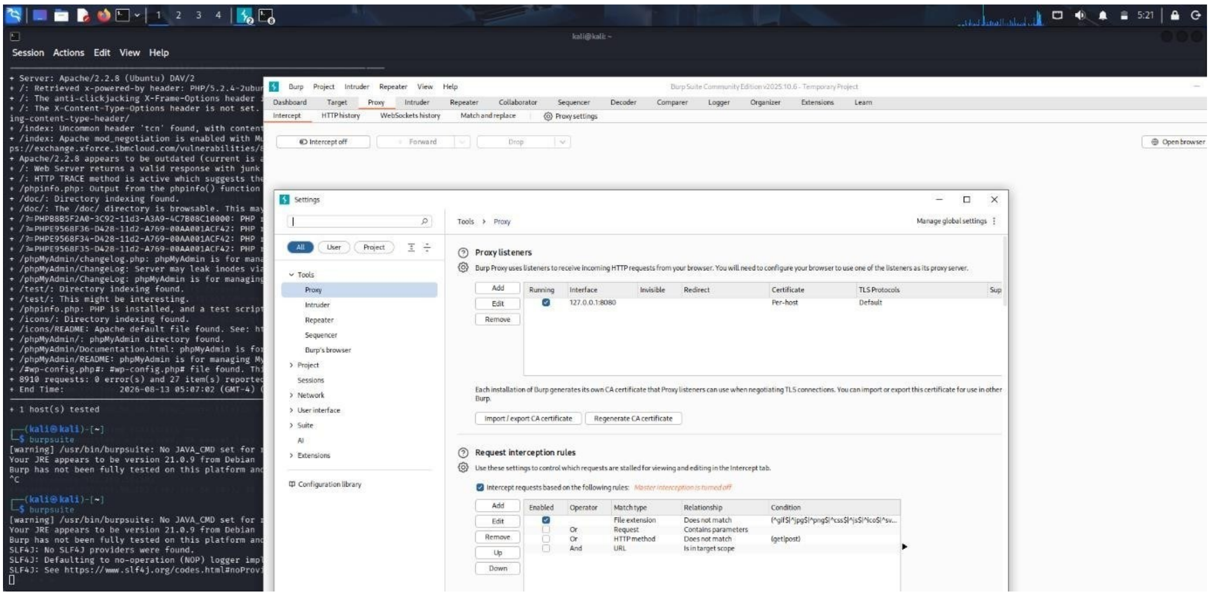
- nikto -h http://192.168.56.102



4. **DVWA Security Configuration :** The security level was configured to Low for controlled laboratory testing.

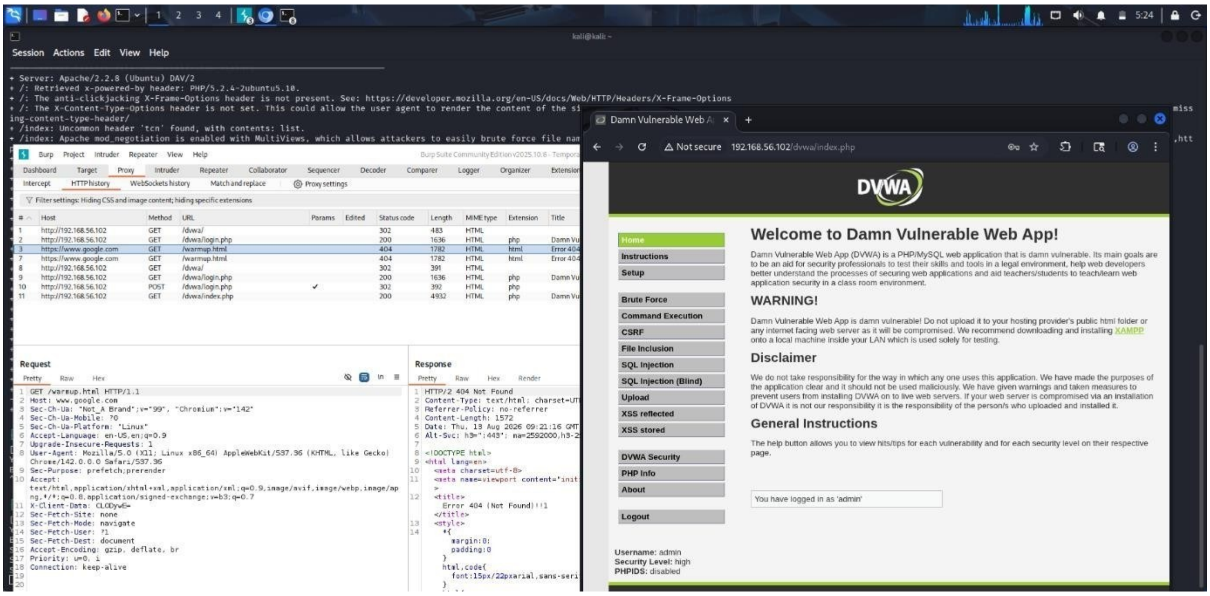


5. **Burp Suite Proxy Configuration** : The Burp Proxy listener was configured on 127.0.0.1:8080



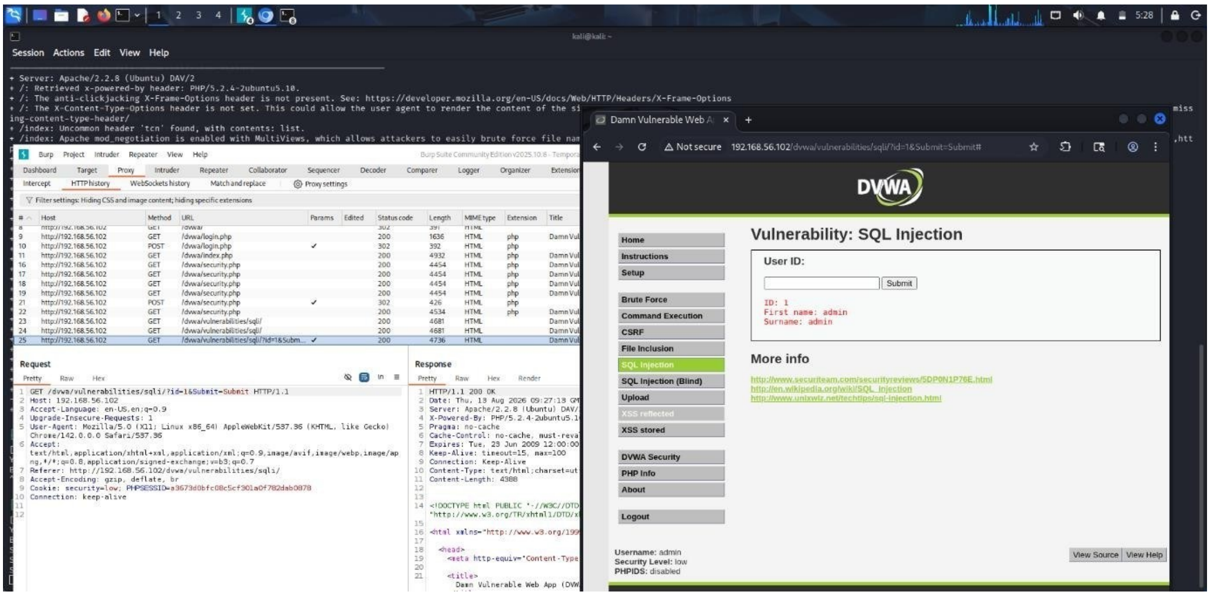
6. Burp Suite HTTP History

- DVWA requests were captured in Burp Suite HTTP History.
- GET and POST requests were observed.
- Login and application requests were recorded.



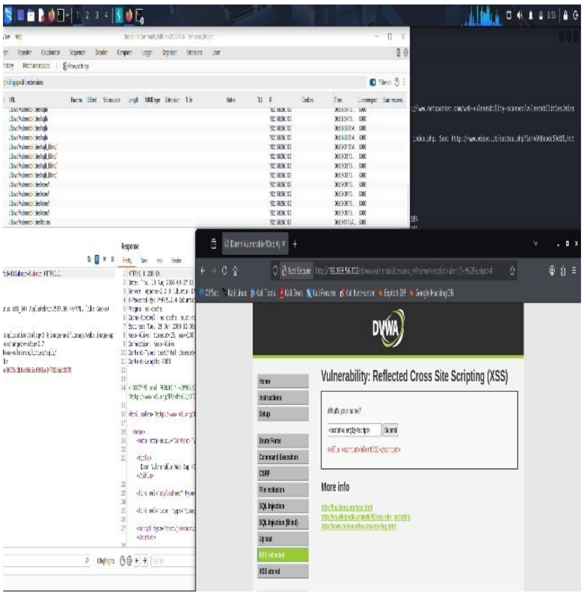
7. SQL Injection Testing

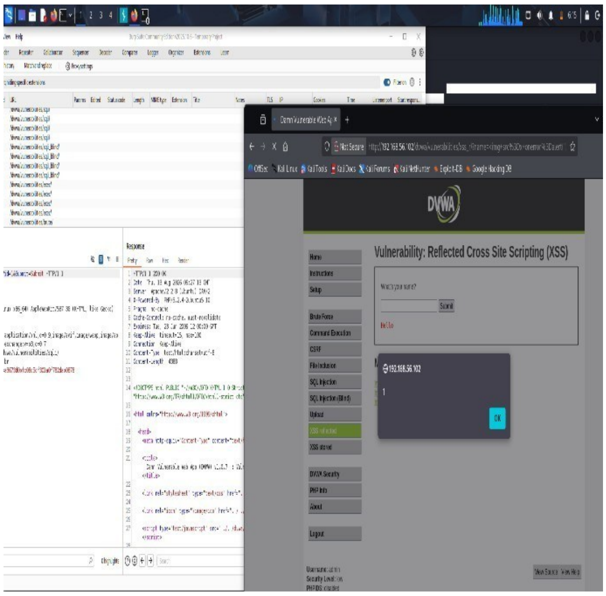
- DVWA SQL Injection module was opened.
- The id parameter was submitted.
- The request was captured using Burp Suite.



8. Reflected Cross-Site Scripting (XSS)

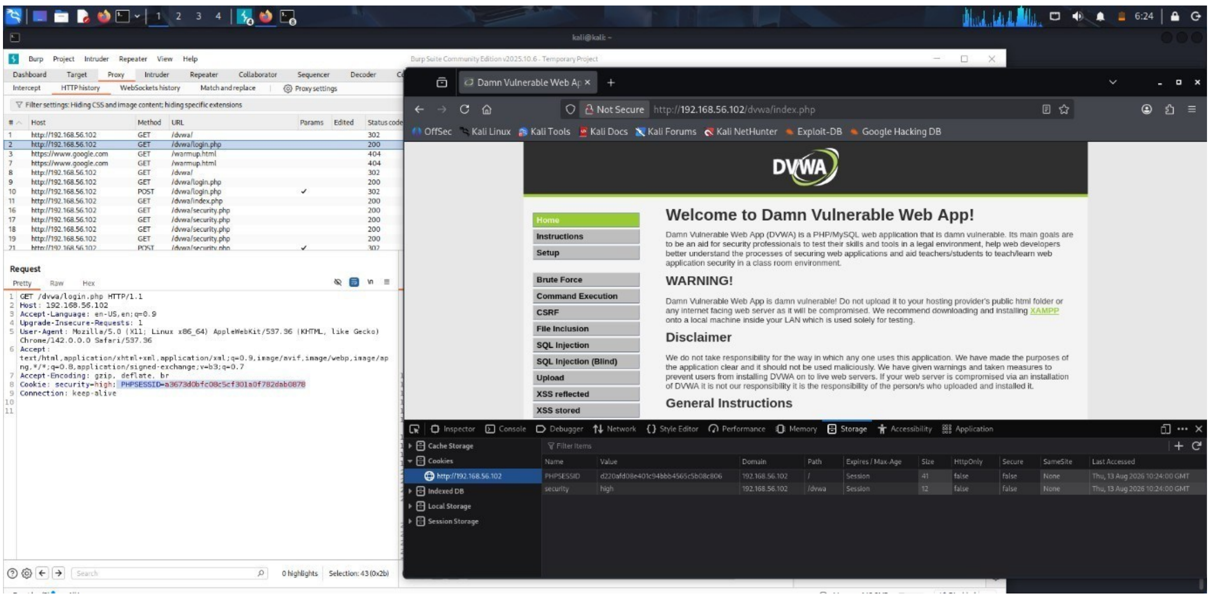
- DVWA Reflected XSS module was accessed.
- A JavaScript test input w as submitted.





9. Session Cookie Analysis

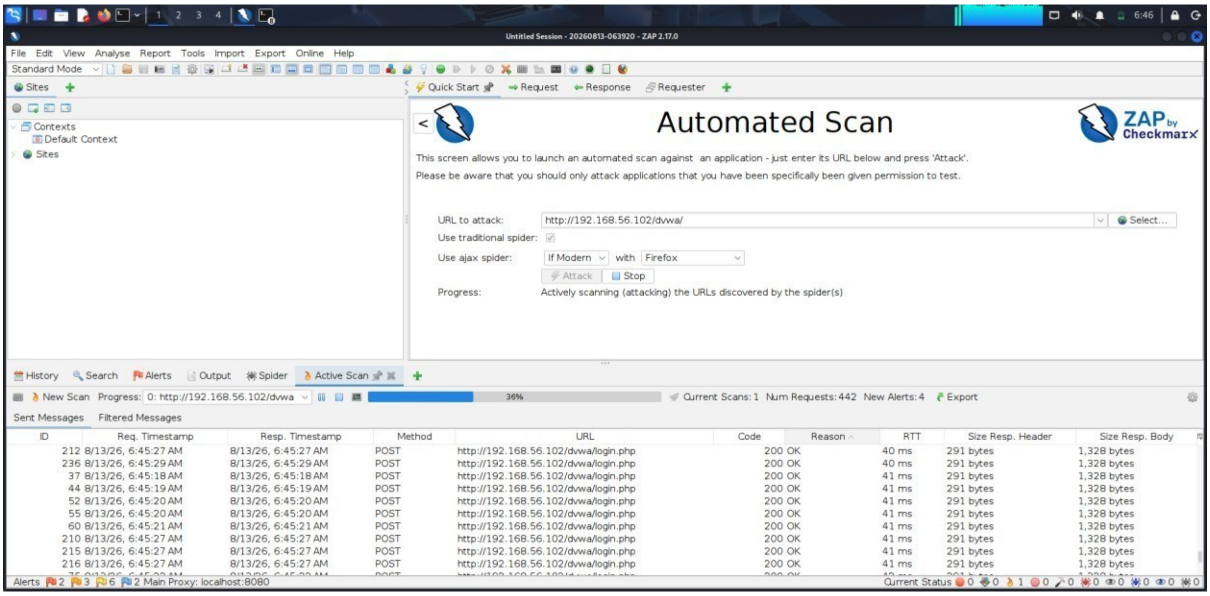
- Browser Developer Tools were used to inspect application cookies.
- The PHPSESSID session cookie was identified.
- Cookie security attributes were examined.



10. OWASP ZAP Automated Scan

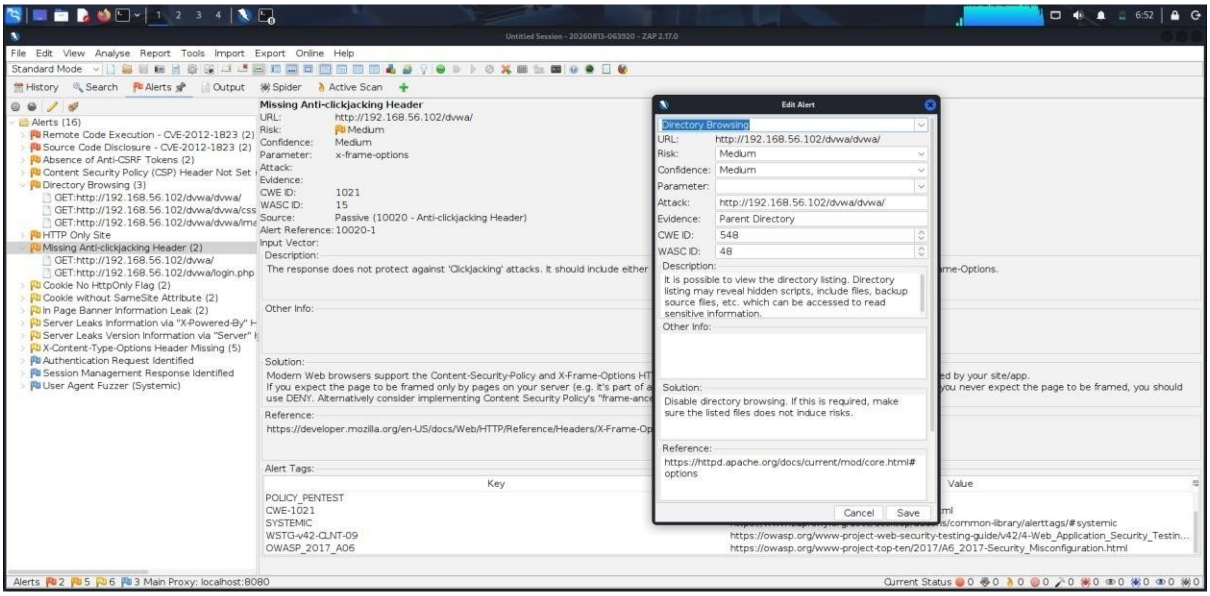
Target : http://192.168.56.102/dvwa/

- 442 requests generated.
- 4 new alerts detected at that stage.



11. OWASP ZAP Security Findings :

- Directory Browsing.
- Missing Anti-clickjacking Header.
- Missing Anti-CSRF Tokens.
- Content Security Policy Header Not Set.



12. Conclusion :

The vulnerability assessment was successfully completed in the controlled DVWA/Metasploitable laboratory environment. The results demonstrate the importance of secure server configuration, input validation, HTTP security headers, sessioncookie protection, and regular automated vulnerability scanning.

Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and XContentType-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificaterelated issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

Aim : To evaluate HTTPS, TLS, certificate, and HTTP security-header configuration using Curl and Nmap.

Target: 192.168.56.102 (Metasploitable 2)

Tools: Curl, Nmap

Commands Used :

- curl -I http://192.168.56.102
- curl -k -I https://192.168.56.102
- curl -I http://192.168.56.102:443
- nmap -Pn -p 443 --script ssl-enum-ciphers 192.168.56.102
- nmap -Pn -p 443 --script ssl-cert 192.168.56.102

Practical Observations :

HTTP returned 200 OK, but no HTTP-to-HTTPS redirect was observed. The HTTPS Curl request failed with wrong version number. A request using HTTP on port 443 returned 200 OK, indicating that port 443 was serving HTTP instead of properly negotiating TLS. Nmap detected port 443 as open but could not enumerate TLS ciphers or certificate information.

Findings and Severity

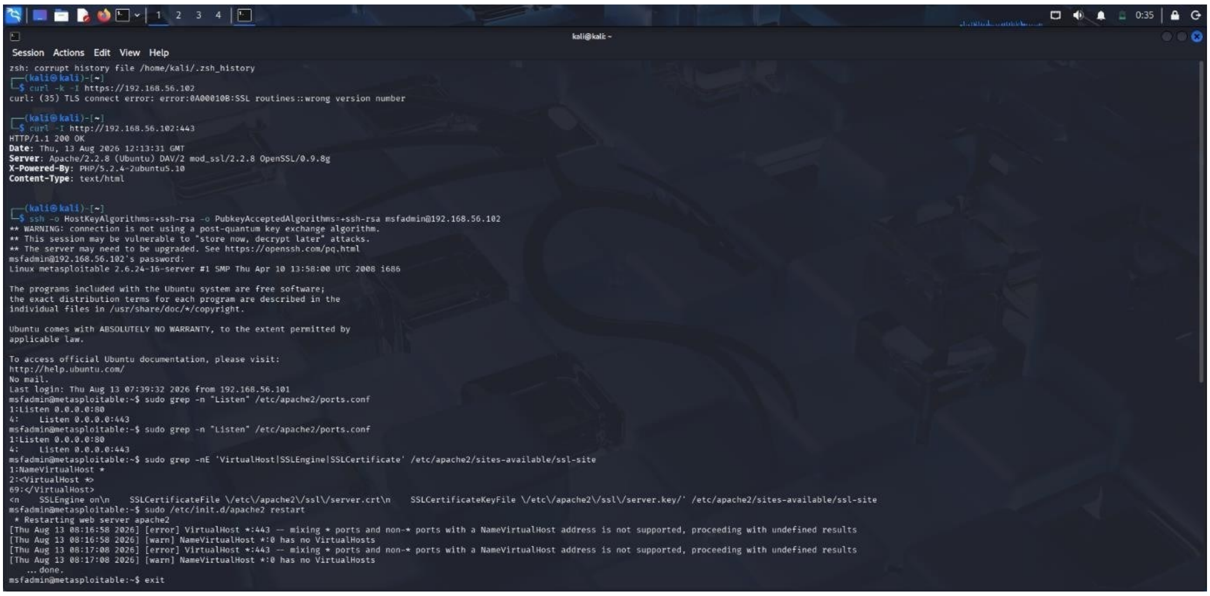
- Missing HTTP-to-HTTPS redirection : Medium
- Missing CSP, HSTS, X-Frame-Options and X-Content-Type-Options : Medium/Low
- Improper HTTPS/TLS configuration on port 443 : High

Remediation

Configure Apache to correctly serve HTTPS on port 443, use valid TLS certificates, redirect HTTP to HTTPS, disable obsolete TLS protocols, use strong cipher suites, and implement CSP, HSTS, X-Frame-Options and X-Content-Type-Options.

Result

The web server's HTTPS and security-header configuration was successfully assessed. Multiple security-hardening deficiencies and an improperly configured HTTPS service were identified.



Question 4: Broken Access-Control Assessment

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

Aim : To use OWASP ZAP to capture and analyze HTTP requests and responses of an authorized vulnerable web application and examine access-control related information.

Tools Used :

- Kali Linux
- OWASP ZAP 2.17.0
- Metasploitable2
- Mutillidae web application

Procedure

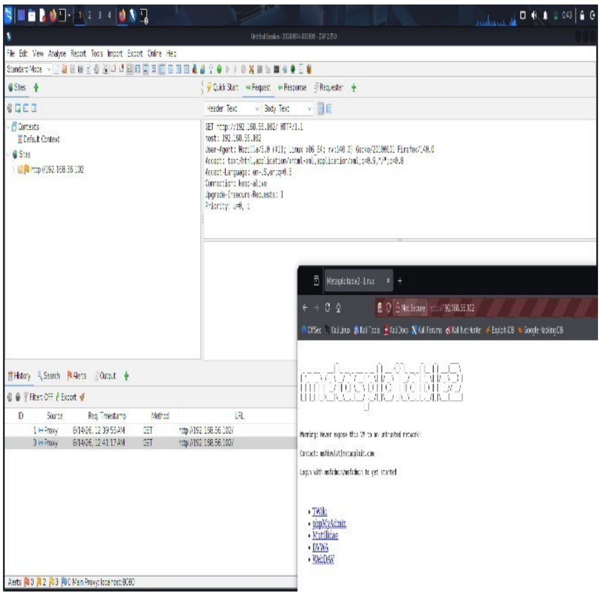
- Started OWASP ZAP in Kali Linux.
- Configured the browser to use ZAP as the proxy.
- Accessed the authorized laboratory server at 192.168.56.102.
- Opened the Mutillidae application.
- Navigated through the application while ZAP captured the generated HTTP requests.
- Opened History in ZAP and examined the captured requests.
- Analyzed request methods, URLs, parameters, cookies/session information and HTTP responses.
- Compared the request and response information to understand how authentication and authorization would be evaluated.
- Checked ZAP Alerts for security issues detected in the intentionally vulnerable application.

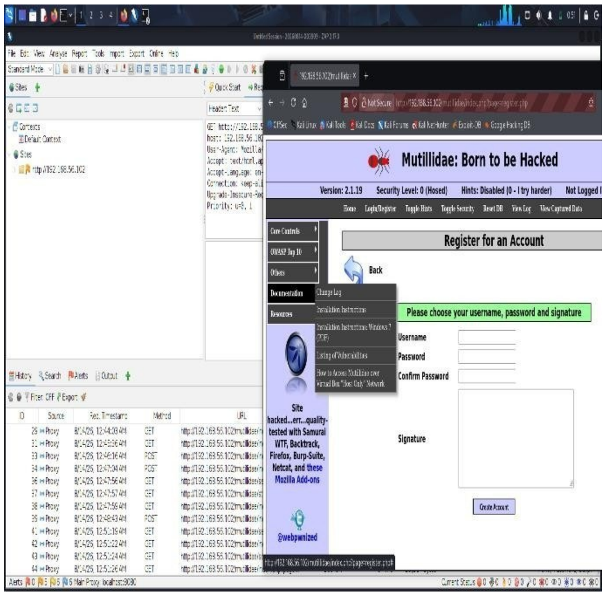
Commands / Configuration

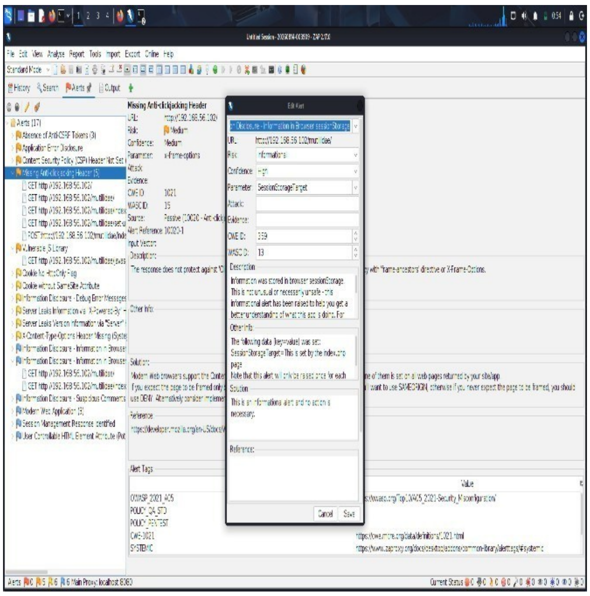
Target: http : //192.168.56.102/

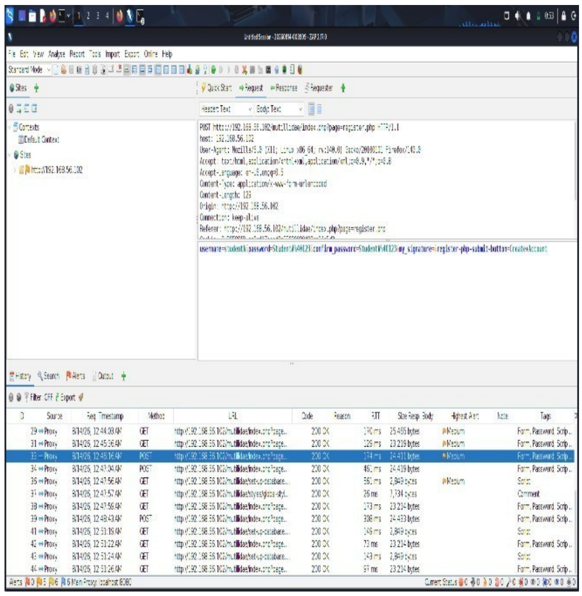
Mutillidae : http://192.168.56.102/mutillidae/

ZAP Proxy: 127.0.0.1:8080









Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

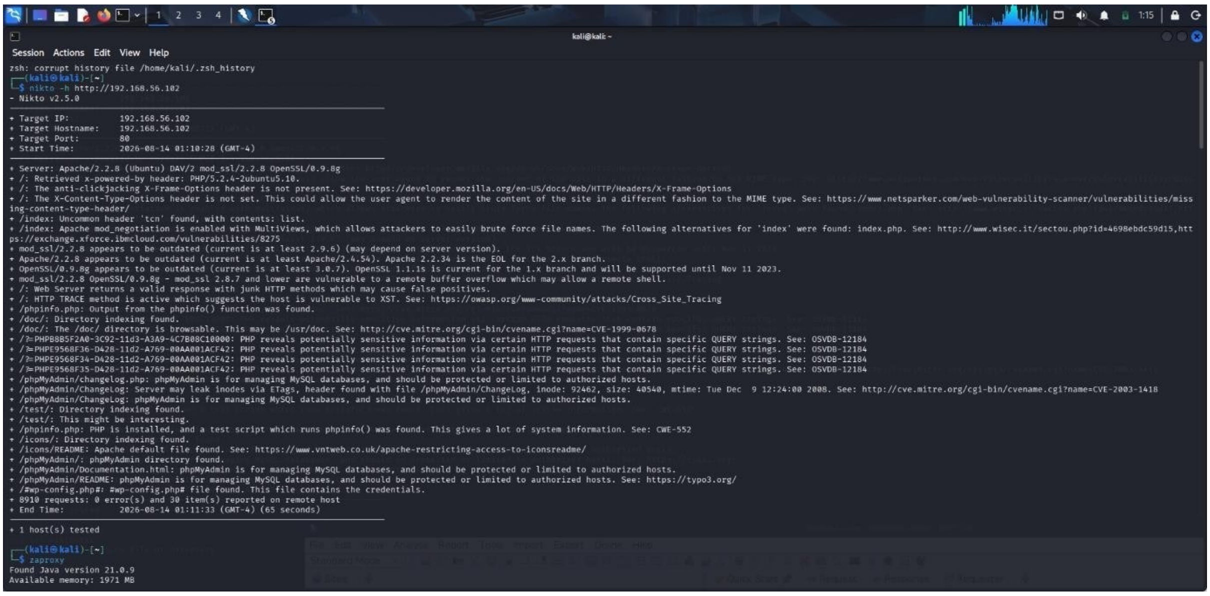
Aim : To identify web-server misconfigurations and web-application vulnerabilities using Nikto and OWASP ZAP, manually validate selected findings, classify their severity, and recommend remediation.

Target and Tools

- Target: 192.168.56.102
- Nikto: Web-server vulnerability scanner
- OWASP ZAP 2.17.0: Web-application security scanner Browser & curl: Manual validation

Nikto identified:

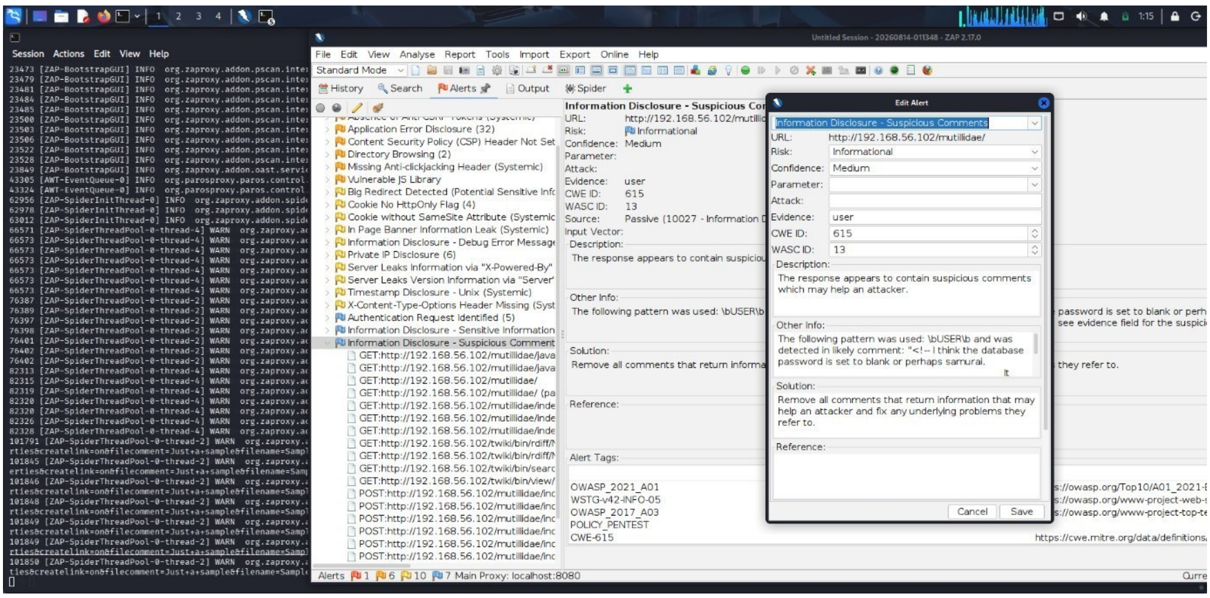
- Apache/2.2.8 and OpenSSL/0.9.8g outdated components
- Missing security headers
- Directory indexing in /doc/, /test/ and /icons/
- phpinfo.php information disclosure
- phpMyAdmin exposure
- Other server information disclosures



OWASP ZAP Results

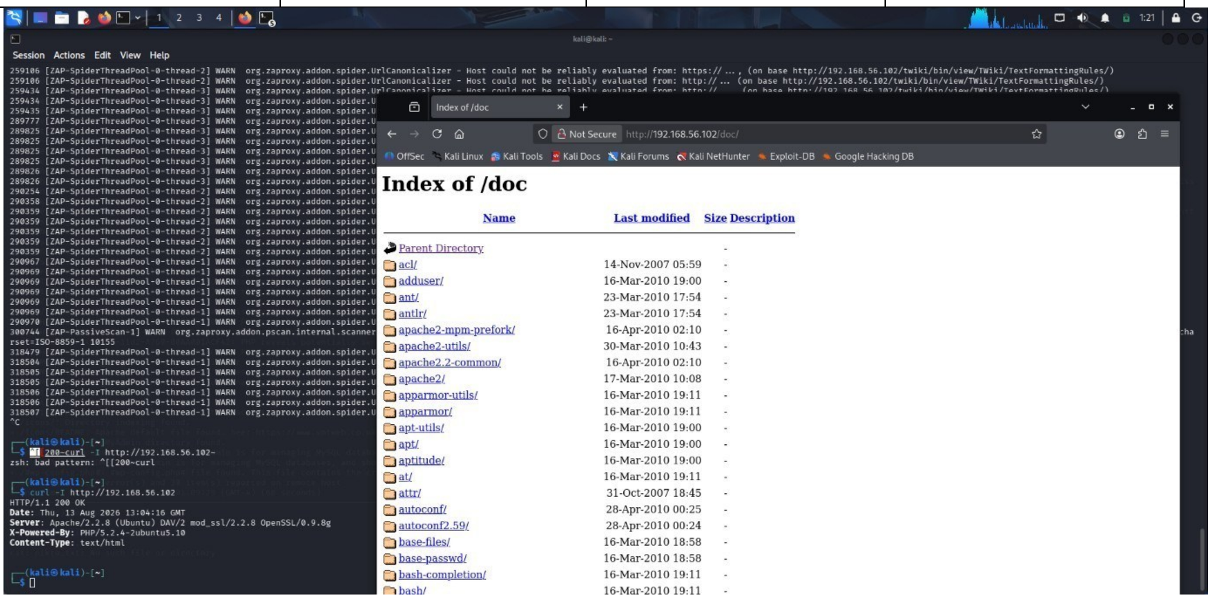
ZAP reported:

- Content Security Policy Header Not Set
- Directory Browsing
- Missing Anti-clickjacking Header
- X-Content-Type-Options Header Missing
- Cookie security issues

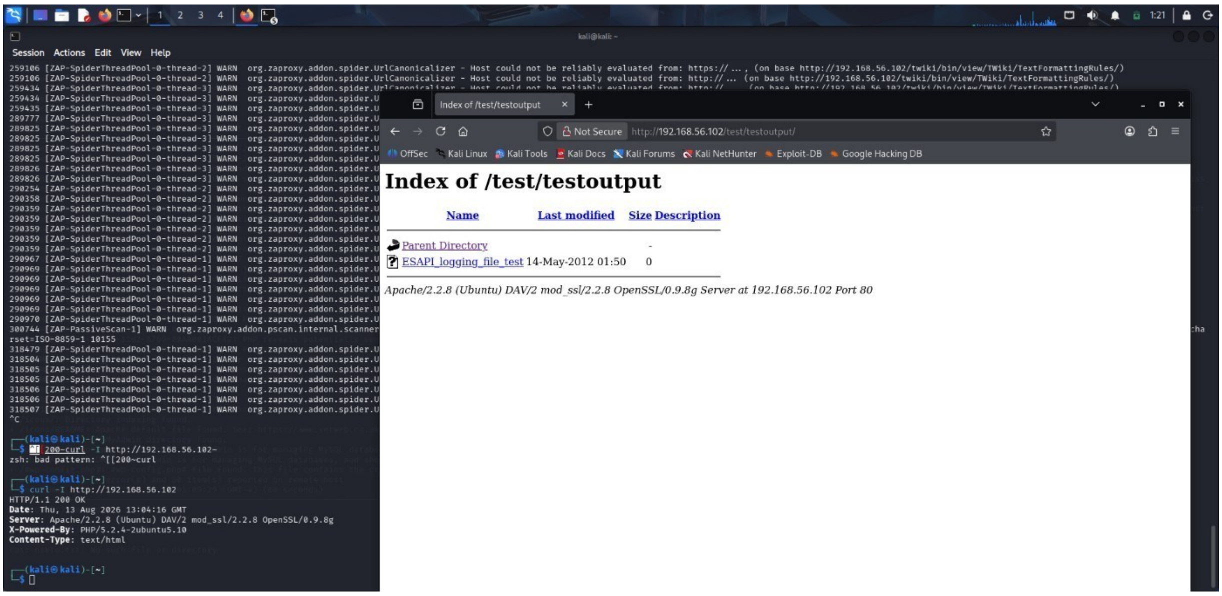


5. Manual Validation :

Finding	Validation Evidence	Result	Severity
Missing security headers	curl -I http://192.168.56.102 showed headers such as CSP/X-Frame-Options absent	Confirmed	Low/Medium
Directory browsing	/doc/ displayed Index of /doc with directory contents	Confirmed	Medium
Information disclosure	Nikto identified phpinfo.php; browser validation performed	Confirmed	Medium



- Nikto provided stronger server-level findings, while ZAP provided broader webapplication and HTTP security findings. Some findings, such as directory browsing and missing security headers, were identified by both tools.



6. Comparison

Tool	Main Purpose	Findings
Nikto	Web-server assessment	Outdated software, directory indexing, phpinfo, server misconfiguration
OWASP ZAP	Web-application assessment	Security headers, information disclosure, cookies, directory browsing and application alerts